🔐 SSL, TLS & Kubernetes Security

📌 What is SSL and TLS?

- Both SSL and TLS are encryption protocols
- TLS (`Transport Layer Security`) is a cryptographic protocol used to secure communication over networks.
- SSL (`Secure Sockets Layer`) Old protocol ❌, Used to secure communication in the past, Replaced by `TLS`

✅ It ensures:

- 🔒 Confidentiality (`encryption`)
- 🧾 Integrity (`no tampering`)
- ✅ Authentication (`identity verification`)

🔐 TLS (Transport Layer Security)

- ✅ Modern & secure version of SSL, Fixes SSL vulnerabilities
- Used in `HTTPS` , `APIs` , `emails` , `Kubernetes` (🔁 Istio mTLS Encrypts `service-to-service traffic` , Ingress, etc.)
- Current versions: `TLS 1.3` (latest & fastest)

🌐 Real Example:

Real TLS Handshake Flow (Step-by-Step)

- TLS Handshake is the process of establishing a secure connection between a `client` and a `server`

👉 When you open a website:

1. Browser: “Who are you?”
2. Server: “Here’s my certificate”
3. Browser: “Looks legit (browser has successfully verified the server’s certificate) 👍” (“I trust this server’s identity.”)
4. Both: “Let’s use this secret key”

5. Start secure communication

```
Opening https://google.com                    # (Using Google as example)

🔐 What actually happens (real life)
1️⃣ In Chrome browser You type URL :
     https://google.com
              - 👉 Your browser says: “Hey Google, I want to connect securely”

2️⃣ Google sends certificate 📜
    Google sends its TLS certificate It’s like:
      - 👉 “Here is my ID proof, I am really Google”

3️⃣ Browser checks it 🔍
   Chrome checks:
     - Is it really from Google?
     - Is it signed by trusted authority?

👉 If valid → continue, 👉 If not → ⚠️ “Not Secure”

4️⃣ Secret key is created 🔑
   - Your browser + Google create a secret password
   - Example: Secret Key = 7X@kP9!                    # 👉 No one else can

5️⃣ Now data is encrypted 🔒
   - When you search: "DevOps tutorial"

👉 It becomes something like: A9#kLm!2xZ
   - Sent over internet (safe) and Google decrypts it back

🚀 Start Secure Communication
👉 (Encrypted Data Transfer)
   All data now:
     - 🔒 Encrypted
     - ⚡ Fast (symmetric encryption)
```

TLS ensures:

- 🔒 Encryption
- 🛡️ Data protection
- 🔐 Shows padlock in browser
- 🔑 Secure passwords & payments

“TLS uses asymmetric encryption for the handshake and symmetric encryption for data transfer to balance security and performance.”

🔒 Where TLS is Used?

- HTTPS websites 🌍
- Secure emails (SMTPS, IMAPS) 📧
- VPNs 🔐
- APIs ⚙️
- Kubernetes (etcd, API server) ☸️

🔐 TLS in Kubernetes

TLS Provides 3 Security Pillars:

- 🔒 Confidentiality (`Data is encrypted`)
- 🗒️ Integrity (`no tampering`)
- ✅ Authentication (`identity verification`)

🚀 Why TLS is Used in Kubernetes?

- 🔐 Secure communication between components (`API server` , `etcd` , `kubelet`)
- 🕵️ Prevent Man-in-the-Middle (`MITM`) attack 🚫
- ✅ Authentication (who is connecting) and `Authenticate services`
- 🔐 Secure `kubectl → API server`

🕵️ What is a Man-in-the-Middle (MITM) attack?

A Man-in-the-Middle attack happens when an attacker secretly sits between you and a website 🔓 What attacker can do :

- 👀 Read your data (`passwords` , `OTPs` , `cards`)
- ✏️ Modify data (change transactions)
- 🎭 Create `fake login` websites pages

🔑 TLS Certificates in Kubernetes

🧩 Category	📌 Details	💡 Explanation
🔐 Certificate Type	📜 X.509 Certificates	Standard format used to verify identity in Kubernetes

🧩 Category	📌 Details	💡 Explanation
📂 `.crt` File	📄 Certificate	🪪 Public identity of server (shared with others)
🔑 `.key` File	🔒 Private Key	🚫 Secret key used for encryption (must be protected)

Kubernetes uses X.509 certificates

📂 Files:

- `.crt` → Certificate `.key` → Private key

🛠️ Managed by:

- kubeadm
- cert-manager
- OpenSSLL

🧩 Tool	📌 Type	💡 Description
🛠️ kubeadm	⚙️ Cluster setup tool	🤖 Automatically generates certificates during cluster initialization. (creates certs when cluster starts)
🤖 cert-manager	🔄 Certificate controller	🔁 `Auto-creates & renews TLS certificates` inside cluster
🧰 OpenSSL	🛠️ CLI tool	✍️ Used to manually generate certificates and keys (create your `own certs`)

🔐 TLS Usage in Kubernetes

👉 TLS in Kubernetes = Secure communication everywhere (user ↔ cluster ↔ internal services) 🔐

🧩 Component	📌 Where Used	💡 Explanation
🔗 API Server	🌐 `kubectl → API Server`	🔒 All API requests (`kubectl` , `controllers`) are encrypted and authenticated (TLS secured)
💾 etcd	🗄️ Database communication	🔐 Stores cluster data securely (`encrypted`) so attackers can't read them

🧩 Component	📌 Where Used	💡 Explanation
🖥️ kubelet ↔ API Server	🤝 Secure worker Node communication	🔐 Uses mTLS for mutual authentication (🤝 `Secure communication`)
⚙️ Controller Manager & Scheduler	🔄 Control plane components	🛡️ Secure communication with API Server
🌐 Ingress	🌍 External traffic	🔒 Provides HTTPS (TLS) access to applications
🔄 Admission Webhooks	🧠 Validation/Mutation	🔐 Secure API calls during resource creation
🕸️ Service Mesh (`Istio mTLS`)	Pod ↔ Pod communication	🔐 Encrypts service-to-service traffic inside cluster
🤖 cert-manager	Certificate automation	⚙️ Automatically creates & renews TLS certificates
🔌 Webhook TLS	Control-plane communication	🔒 Ensures secure internal `API extensions`

🔄 TLS vs mTLS

🧩 Feature	🔐 TLS	🔐🔐 mTLS (Mutual TLS)
🪪 Authentication	👤 Server only verifies itself	🤝 Both `Client + Server` verify each other
🛡️ Security Level	✅ High	🔥 Very High (`double verification`)
🌐 Usage	🌍 HTTPS (`browser → server`)	🕸️ Service Mesh (e.g., `Istio`)

🔐 Kubernetes TLS Certificate Files

📂 File	🧩 Type	💡 Purpose
📄 `.crt`	📜 Certificate	🪪 Public identity of server (shared with clients)
🔑 `.key`	🔒 Private Key	🚫 Secret key used to encrypt/decrypt data
🏢 `ca.crt`	🏛️ Certificate Authority	✅ Used to verify if certificate is trusted

📂 Certificate Files Paths (kubeadm) :

📍 Path	🧩 Component	💡 Explanation
`/etc/kubernetes/pki/apiserver.crt`	🌐 API Server Cert	🪪 Identity of Kubernetes API Server
`/etc/kubernetes/pki/apiserver.key`	🔑 API Server Key	🔒 Private key for API Server
`/etc/kubernetes/pki/ca.crt`	🏛️ CA Certificate	✅ Used by all components to trust each other

⚡ Real Flow

- 👉 .crt = Who you are (`ID`)
- 👉 .key = `Secret password` 🔒
- 👉 ca.crt = Trusted authority (`verifies ID`)
- 👉 API Server shows `apiserver.crt`
- 👉 Client verifies using `ca.crt`
- 👉 Secure connection established using `apiserver.key` 🔐

🤖 What is cert-manager?

cert-manager is a Kubernetes controller that:

- Issues TLS certificates 🔐
- Auto-renews certificates before expiry 🔄
- Stores in as Kubernetes Secrets 📦

🛠️ Manual TLS Certificate Creation

🔐 1. Generate Certificate (OpenSSL)

```
openssl genrsa -out myapp.key 2048
```

```
openssl req -new -key myapp.key -out myapp.csr -subj "/CN=myapp.example.com"

openssl x509 -req -in myapp.csr -signkey myapp.key -out myapp.crt -days 365
```

🔐 Create Kubernetes TLS Secret

```
kubectl create secret tls myapp-tls-secret \
--cert=myapp.crt \
--key=myapp.key \
--namespace=default
```

🌐 Use TLS in Ingress

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: myapp-ingress
spec:
  tls:
  - hosts:
    - myapp.example.com
    secretName: myapp-tls-secret
```

🌐🔐 TLS in Kubernetes Gateway API (Terminate vs Passthrough)

👉 "In Gateway API, TLS is configured at the Gateway listener level, not in the route. Routes only handle traffic routing."

🧩 Feature	🔓 Terminate (Gateway API)	🔐 Passthrough (Gateway API)
📍 TLS handled at	🌐 Gateway (Listener)	🖥️ Backend (Service/Pod)
🔄 Traffic inside cluster	📡 HTTP (via `HTTPRoute`)	🔒 HTTPS (via `TLSRoute`)
🛡️ Security	✅ High	🔥 Very High (end-to-end encryption)
⚙️ Complexity	👍 Easy	⚠️ Advanced

🧩 Feature	🔓 Terminate (Gateway API)	🔐 Passthrough (Gateway API)
📜 Certificate location	📂 Gateway (`certificateRefs`)	📦 Backend application
🧭 Route Type	🌍 `HTTPRoute`	🔐 `TLSRoute`
🔧 TLS Mode	`Terminate`	`Passthrough`

⚡ When to use

- 🔓 Termination → ✅ (Most Common) `TLS ends at Gateway` , Traffic becomes HTTP internally (`decrypt early`)
    - 🎯 Use Case : Standard web apps (`React` , `APIs` , `microservices`)
- 🔐 Passthrough → ✅ (Advanced) TLS is NOT terminated at Gateway, Encrypted traffic is passed directly to backend
    - 🎯 Use Case : `Banking / highly secure apps` (mTLS setups, zero-trust, strict security) (stay `encrypted end-to-end` 🔒)
    - 📌 Key Points : End-to-end encryption🔥 --Gateway cannot inspect traffic -- Requires backend to manage certificates (Backend service handles `TLS decryption`)

```yaml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: myapp-gateway
spec:
  gatewayClassName: nginx
  listeners:
  - name: https
    protocol: HTTPS
    port: 443
    hostname: myapp.example.com
    tls:
      mode: Terminate    ## 🔓 TLS termination here
      certificateRefs:
      - kind: Secret
        name: myapp-tls-secret
```

🔄 Flow

```
User 🔒 → Gateway (decrypt) → HTTP → Service
```

🧪 Testing

```
curl https://myapp.example.com --insecure
```

📊 TLS Certificate Management Comparison

Method	Production	Auto Renew	Scalable	Trusted
cert-manager	✅ Yes	✅ Yes	✅ Yes	✅ Yes
DigiCert / CA	✅ Yes	❌ Manual	✅	✅
OpenSSL Manual	❌ Limited	❌ No	❌	❌

📂 Certificate Types

🧩 Type	📌 Description	🧠 How It Works	🌐 Example
✅ Single Domain	Covers one domain	👉 Valid only for one exact domain	`example.com`
🌟 Wildcard	Covers all subdomains	👉 Uses `*` to match subdomains	`*.example.com`
🌍 Multi-domain (SAN)	Covers multiple domains	👉 Uses SAN (Subject Alternative Name)	`example.com`, `app.org`

🧠 Simple Understanding

- 👉 Single Domain = Works for only one site, ❌ Not for subdomains
- 👉 Wildcard = Covers all subdomains, Example: `api.example.com`, `blog.example.com`
- 👉 Multi-domain (SAN) = One certificate for multiple different domains

---

✨ Summary

TLS is:

- 🔐 Core of internet security
- ☸️ Critical in Kubernetes
- 🤖 Automatable with cert-manager

- 🚀 Essential for modern apps

⚡ Final understanding

- 👉 TLS is just: Verify identity → ✅ Create secret key 🔑 → Send encrypted data 🔒
- Secure your apps = Secure your users 💙